# Final Presentation

## ASE285 ChatGPT Clone

By: Nabou Diouf

# Project Overview

- Full-stack ChatGPT-style application
- Real-time messaging with AI responses
- User authentication and private sessions
- File upload with AI processing
- Docker-based build and run setup

# Core Requirements

- Users can sign up, log in, and have private chat sessions
- Users can start, manage, and switch between chats
- Users can send messages and receive real-time AI responses
- Conversation history persists across sessions
- Users can upload files shown as clean previews
- App is runnable using Docker outside localhost setup

# Technology Stack

- **Frontend:** React
- **Backend:** Node.js + Express
- **Real-time:** Socket.IO
- **Database:** MongoDB
- **Authentication:** JWT + bcrypt
- **AI:** OpenAI API
- **Deployment:** Docker + Docker Compose

# System Architecture

```
React Frontend
      |
REST API + Socket.IO
      |
Node / Express Backend
      |
MongoDB + OpenAI API
```

- Frontend handles the user interface
- Backend handles auth, sockets, file processing, and AI requests
- MongoDB stores user-specific sessions

# Sprint 1 Progress

- **Week 5:** UI functionality and improvement
- **Week 6:** WebSocket connection and session management
- **Week 7:** End-to-end messaging
- **Week 8:** OpenAI integration

# Sprint 2 Progress

- **Week 10:** Backend persistence for conversations
- **Week 11:** Search, filter, and sort conversations
- **Week 12:** AI thinking indicator
- **Week 13:** Authentication
- **Week 14:** File attachment
- **Week 15:** Tests and deployment

# Metrics

- **Total LoC:** ~3500
- **Features Planned:** 11
- **Features Completed:** 11
- **Test Coverage:** Unit, Integration, Regression, Acceptance

# Testing Summary

- **Unit Tests:** Component-level tests
- **Integration Tests:** Dashboard, socket events, and chat workflows
- **Regression Tests:** Previously fixed bugs
- **Acceptance Tests:** Full user flows

Tests cover authentication, chat sessions, messaging, file uploads, search/filter/sort, and regression checks.

# What Went Wrong

- CORS issues during frontend/backend connection
- Socket.IO had a learning curve
- OpenAI API setup caused delays
- Learning Curve with Docker
- Debugging Docker and MongoDB configuration took time

# What Went Well

- Real-time chat was successfully implemented
- AI integration was completed
- User-specific sessions now work correctly
- File upload works with clean previews
- Docker setup makes the app easier to run

# Key Problems Solved

- Fixed chats showing across different users
- Added JWT-based socket authentication
- Stored conversations by user ID
- Prevented raw file content from displaying in the UI
- Kept file content available for AI processing

# Deployment

The app can be built and run using Docker Compose:

```
docker compose up --build
```

Then open:

```
http://localhost:3000
```

# Future Improvements

- Streaming AI responses
- Drag-and-drop file upload
- Better file preview cards
- Dark/light mode toggle
- Hosted production deployment

# Final Outcome

- Multi-user ChatGPT-style application
- Real-time AI messaging
- Persistent private chat history
- File attachment support
- Search, filter, sort, rename, delete, and pin chats
- Tested and Docker-ready

# Thank You